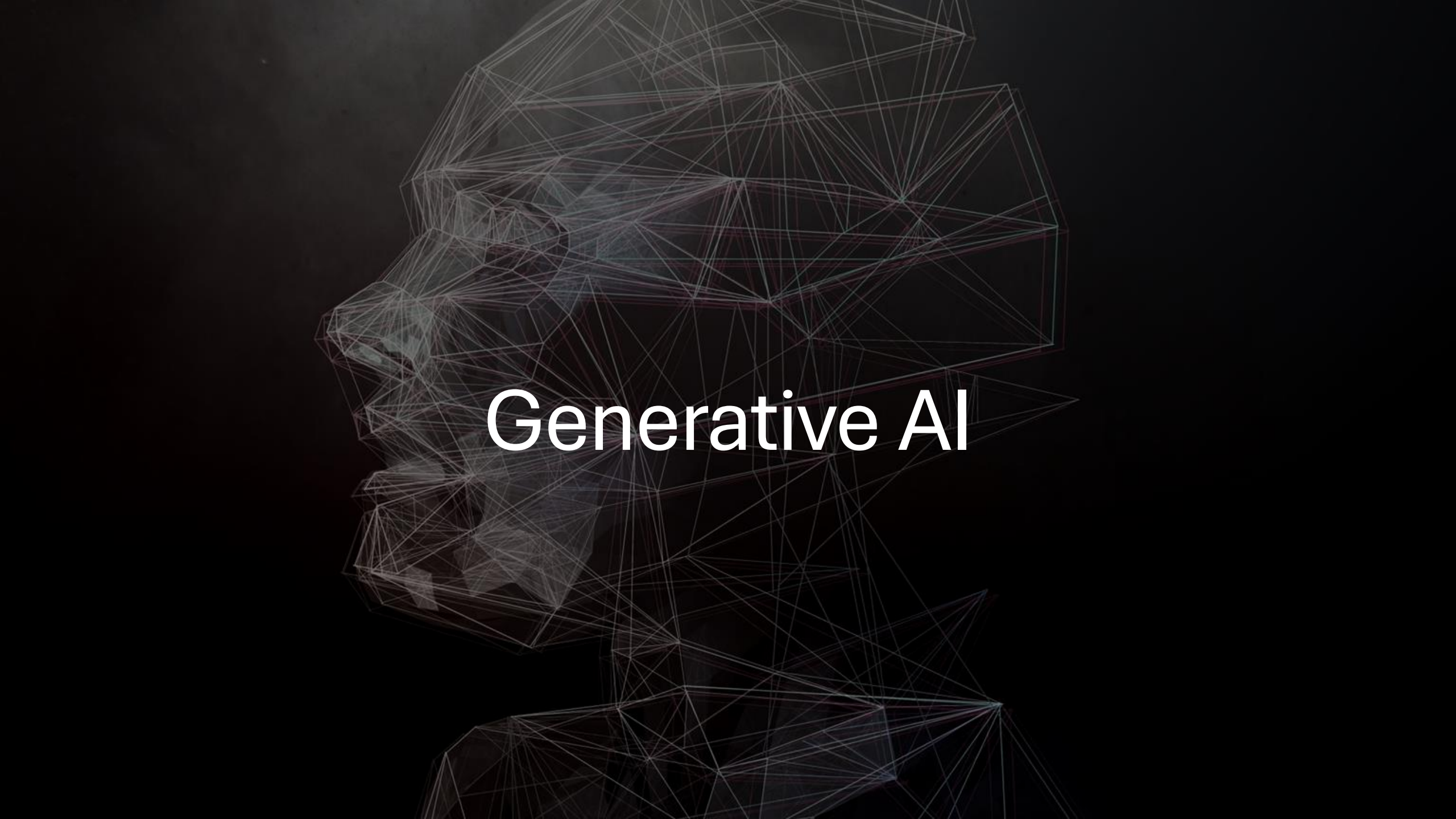
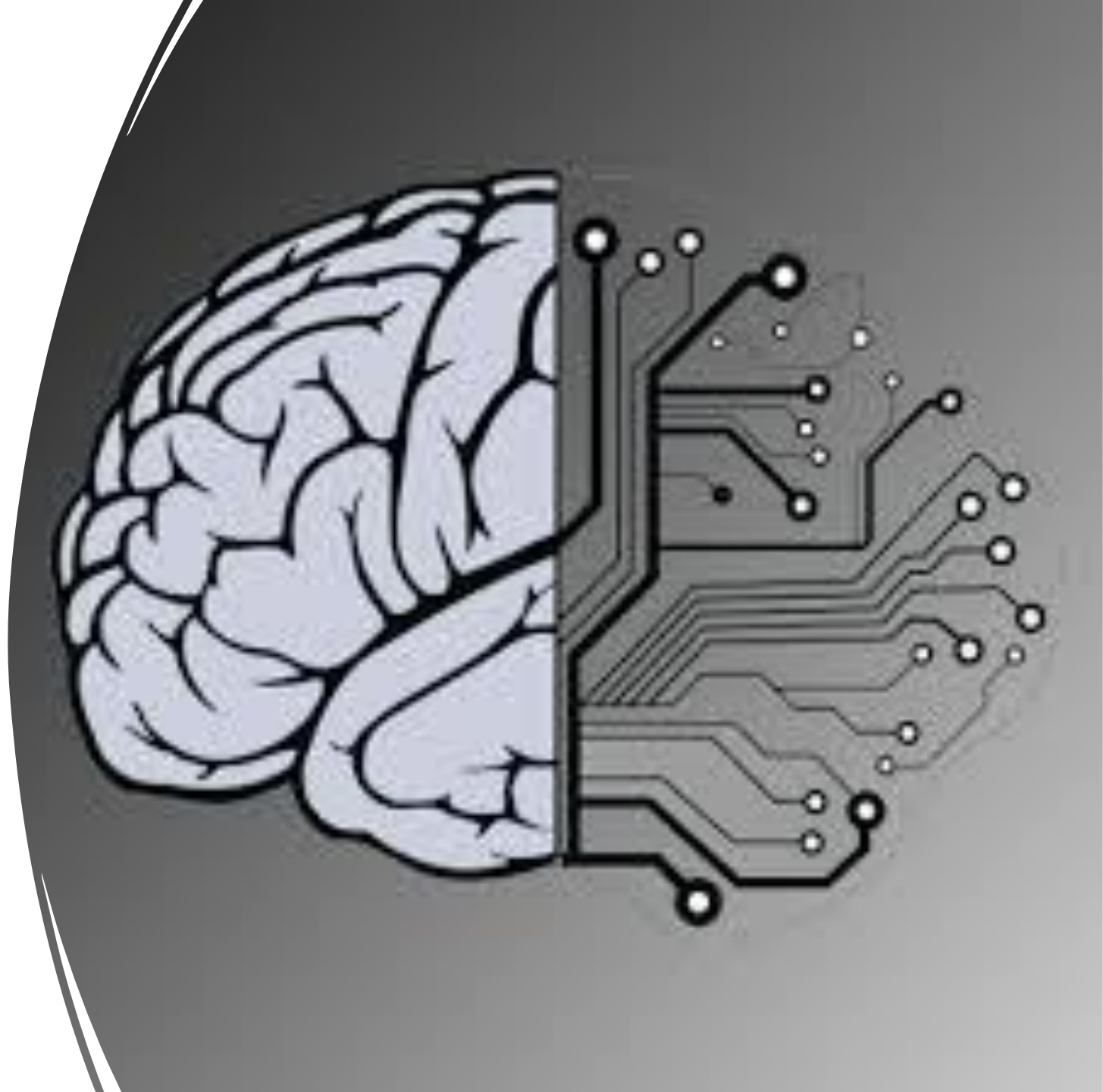


An abstract, complex wireframe structure composed of numerous interconnected lines and polygons, resembling a stylized, multi-faceted object or a network diagram. The structure is rendered in a light gray or white color against a dark, gradient background that transitions from black on the left to a dark gray on the right. The overall effect is one of digital complexity and geometric abstraction.

Generative AI

What is Generative AI

- A type of artificial intelligence that can generate new content, such as text, images, or music.
- **Key Techniques:**
 - Generative Adversarial Networks (GANs)
 - Variational Autoencoders (VAEs)
 - Large Language Models (LLMs)



Applications of Generative AI



Text Generation:

Content creation
Language translation
Summarization



Image Generation:

Art and design
Medical imaging
Video game development



Music Generation:

Composition
Sound effects



Drug Discovery:

Designing new molecules

Image generation



#StyleGAN3 (NVIDIA) Karras, Tero, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. ["Alias-free generative adversarial networks."](#) NeurIPS 2021. [\[code\]](#)

Image generation



Text-to-Image (T2I) generation

“A shot of a 32-year old female, up and coming conservationist in a jungle; athletic with short, curly hair and a warm smile



Text-to-Image (T2I) generation

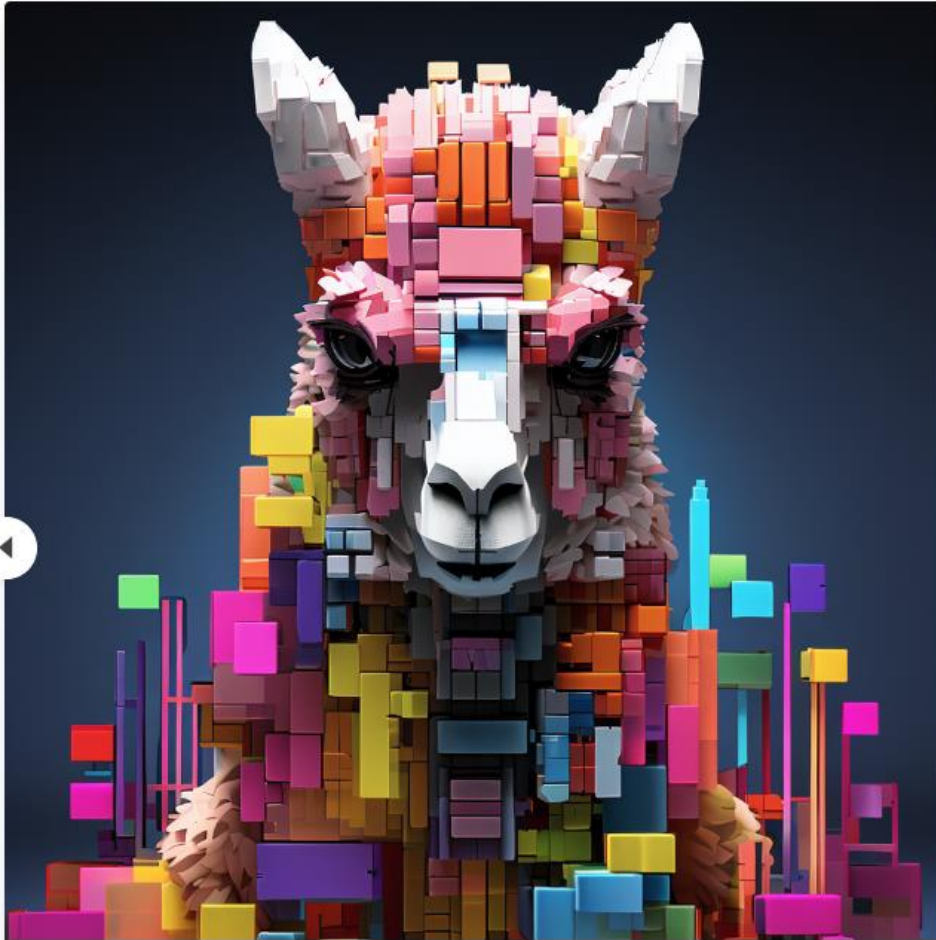
““An expressive oil painting of a basketball player dunking, depicted as an explosion of a nebula.



#DALL-E-2 (OpenAI) Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, Mark Chen ["Hierarchical Text-Conditional Image Generation with CLIP Latents."](#) 2022. [\[blog\]](#)

#DALL-E-3 (OpenAI) James Betker, Gabriel Goh, et al, "Improving Image Generation with Better Captions" 2023 [\[blog\]](#)

Text-to-Image (T2I) generation



A alpaca made of colorful building blocks, cyberpunk.



Real beautiful woman.

Text-to-Music (T2M) generation



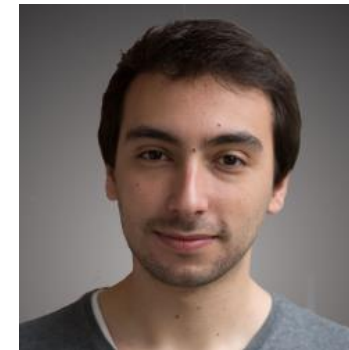
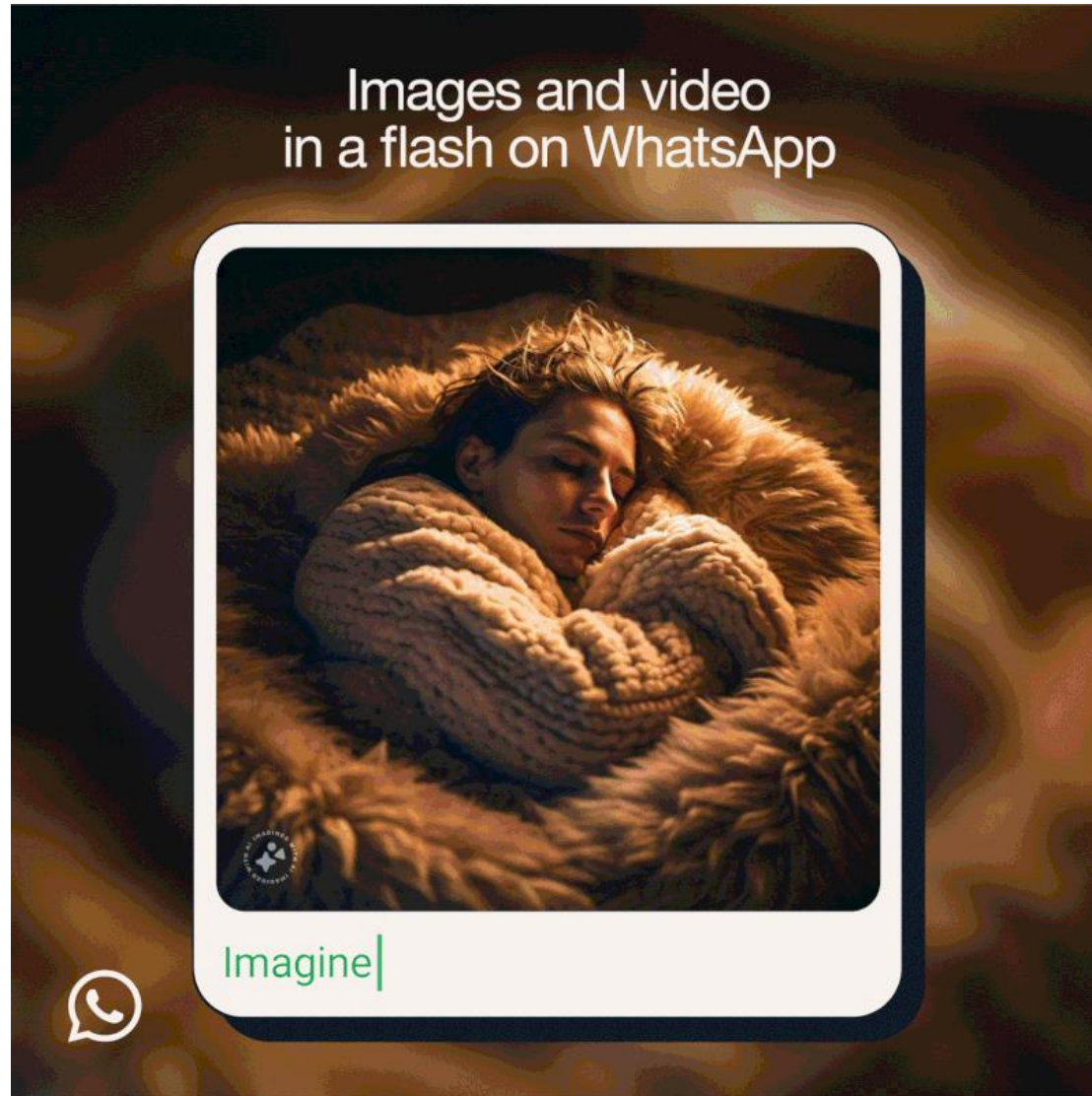
(Stability AI) Evans, Zach, Julian D. Parker, C. J. Carr, Zack Zukowski, Josiah Taylor, and **Jordi Pons**. ["Long-form music generation with latent diffusion."](#) arXiv 2024.

Text-to-Video (T2V) generation



#Sora (OpenAI) Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Wing Yin Ng, Ricky Wang, Aditya Ramesh. [Video generation models as world simulators](#). OpenAI 2024.

Text-to-Image (T2I) generation



Albert Pumarola

[@AlbertPumarola](#)

PhD 2021

Institut de Robòtica Industrial (IRI)
Universitat Politècnica de Catalunya (UPC)

Discriminative Vs Generative Model



Machine learning models can be broadly categorized into discriminative and generative models.



Each type of model has different goals, strengths, and applications.

Discriminative Models

- Discriminative models focus on distinguishing between different classes in the data.
- They model the decision boundary between classes.
- Given input data x , predict the label y .
- Learns the $P(y|x)$
- **Examples:**
 - Logistic Regression
 - Neural Networks
 - Convolutional Neural Networks
- **Advantages:**
 - Often achieve higher accuracy on classification tasks.
 - Directly optimize the decision boundary.

Generative Models

- Generative models focus on modeling the distribution of the data.
- They learn to generate new data points that resemble the training data.
- Model the probability distribution $P(x)$.
- If dataset is labelled, estimates/learn the $P(x|y)$
- **Examples:**
 - Gaussian Mixture Models (GMMs)
 - Hidden Markov Models (HMMs)
 - Variational Autoencoders (VAEs)
 - Generative Adversarial Networks (GANs)
- **Advantages:**
 - Can generate new data samples.
 - Useful for unsupervised learning tasks.

Key Differences

Discriminative

Learn the Decision Boundary

Model $P(y|x)$.

Requires Labelled Data

Generative

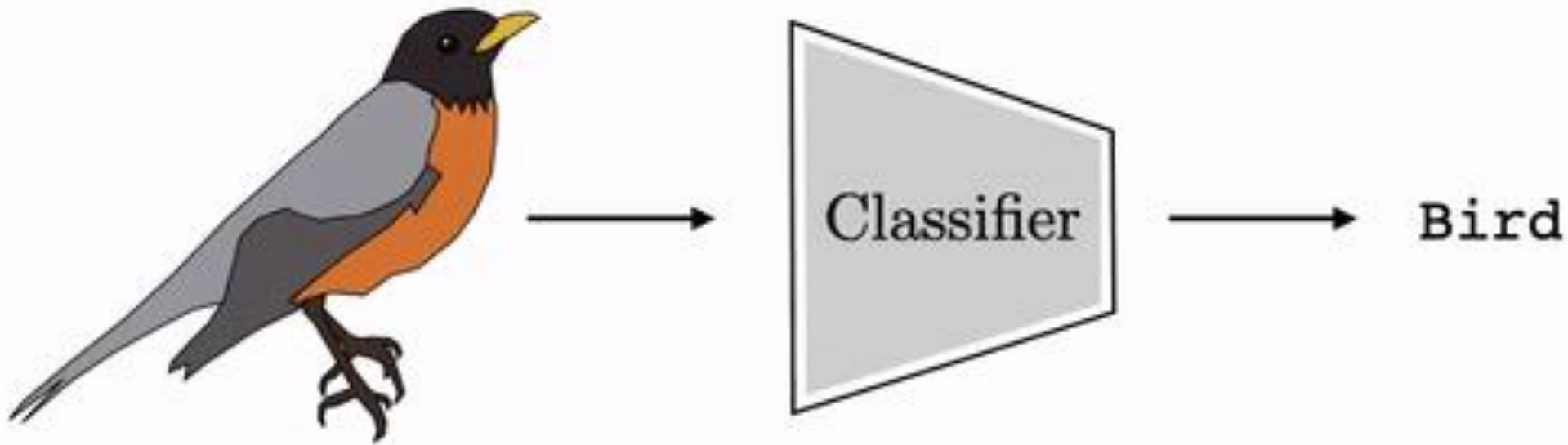
Learn the distribution of the data.

Model $P(x)$.

Can work with unlabeled data.

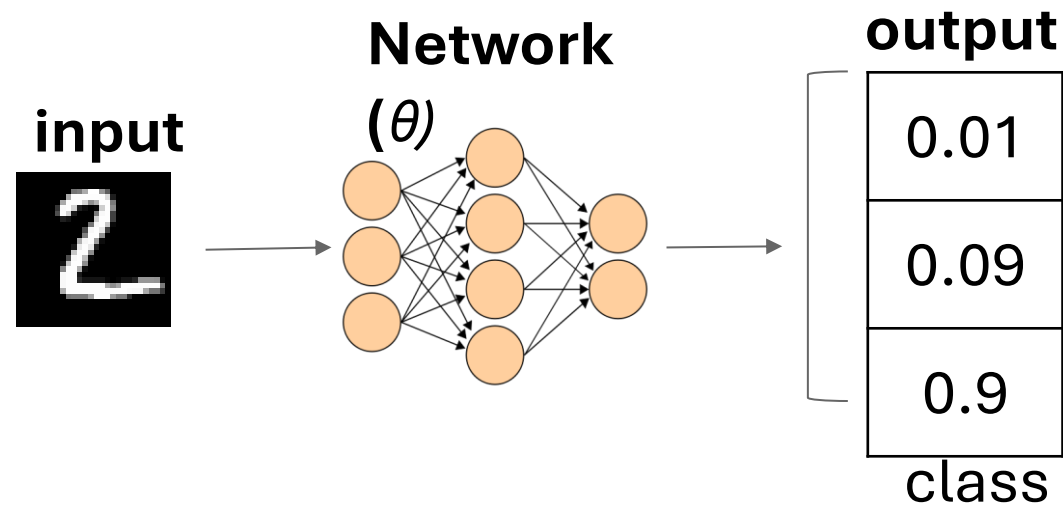
Discriminative vs Generative Models

Philip Isola, [Generative Models of Images](#). MIT 2023.



$P_{\theta}(Y|X)$: Discriminative Models

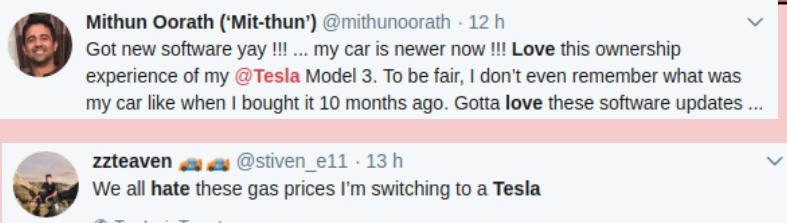
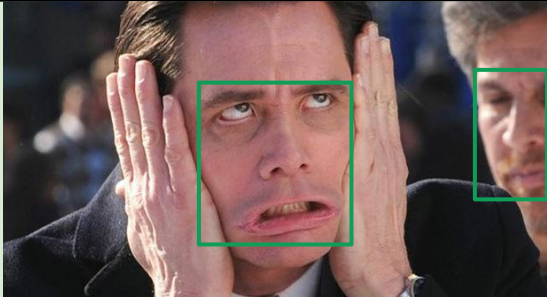
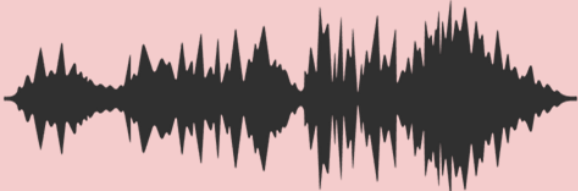
Discriminative model: Tell me the probability of some 'Y' responses given 'X' input.



$$P_{\theta}(Y \mid X = [\text{pixel}_1, \text{pixel}_2, \dots, \text{pixel}_{784}])$$

$P_{\theta}(Y|X)$: Discriminative Models

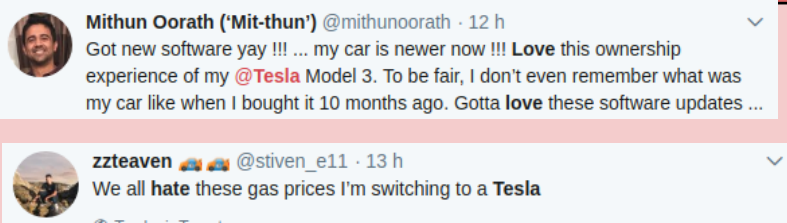
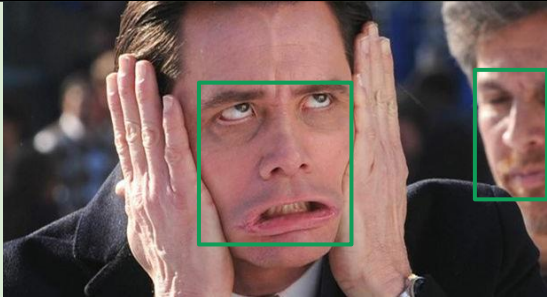
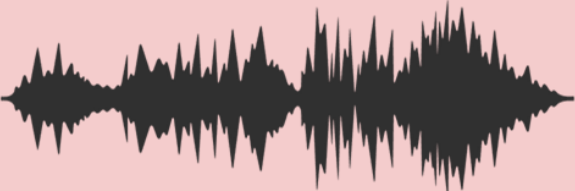
Slide credit:
Albert Pumarola (UPC 2019)

	Classification	Regression
Text		Prob. of being a Potential Customer
Image	 Jim Carrey	
Audio	 What Language?	Speech Translation

Discriminative Modeling
 $P_{\theta}(Y|X)$

X=Data
Y=Labels
 θ = Model parameters

$P_{\theta}(X)$: Generative Models

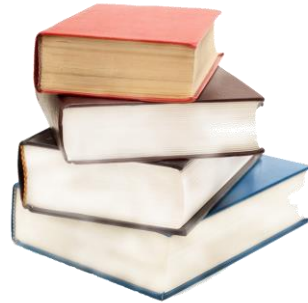
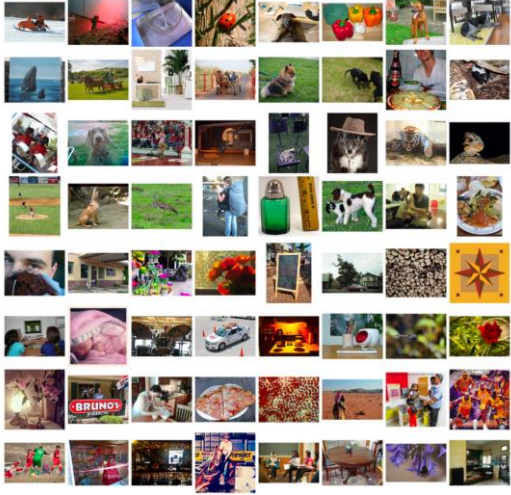
	Classification	Regression	Generative
Text		Prob. of being a Potential Customer	"What about Ron magic?" offered Ron. To Harry, Ron was loud, slow and soft bird. Harry did not like to think about birds.
Image	 Jim Carrey		
Audio	 What Language?	Language Translation	Music Composer and Interpreter MuseNet Sample

X=Data
Y=Labels
 θ = Model parameters

Discriminative Modeling
 $P_{\theta}(Y|X)$

Generative Modeling
 $P_{\theta}(X)$

$P_{\theta}(X)$: Generative Models

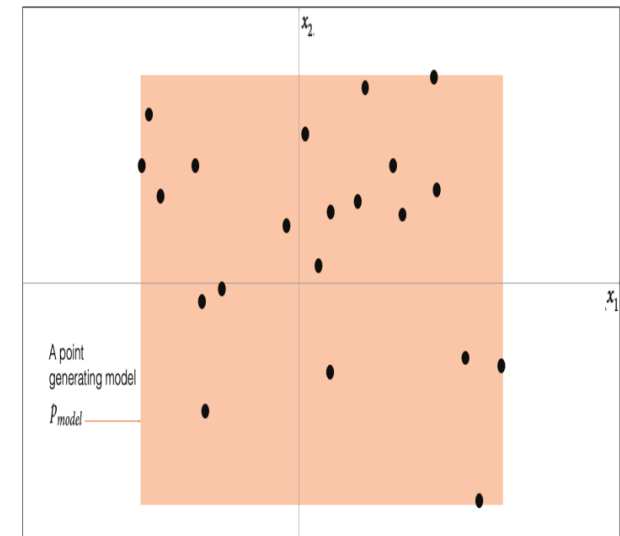
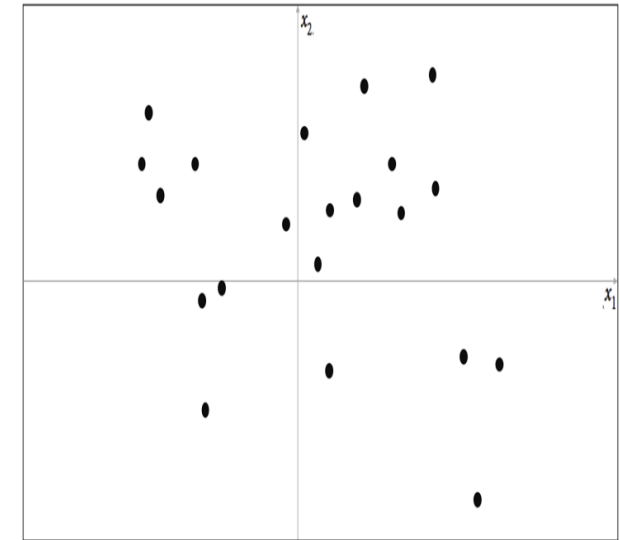


Each real sample x_i comes from an M -dimensional probability distribution $P(X)$.

$$X = \{x_1, x_2, \dots, x_N\}$$

Generative Modeling Framework

- A rule has generated some set of points
- Let's call the rule P_{data}
- Task is to select a point that has been generated by the same rule
- Try to estimate the rule
- Let's call the estimation P_{model}

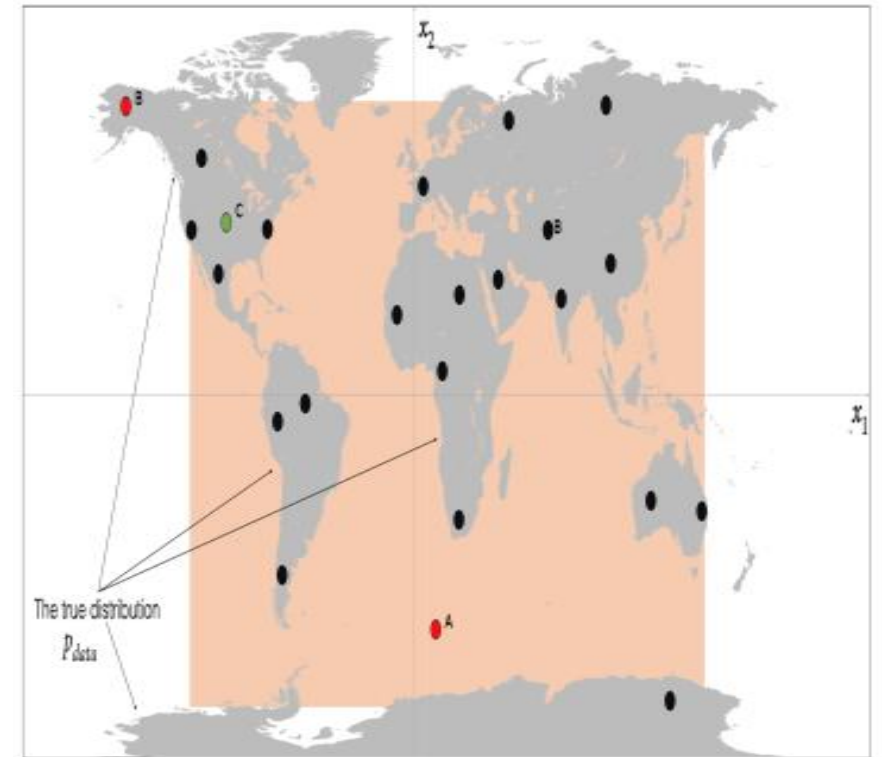


Generative Modelling Framework

- We have a dataset of observations $\mathbf{X}$.
- We assume that the observations have been generated according to some unknown distribution, p_{data} .
- We want to build a generative model p_{model} that mimics p_{data} . If we achieve this goal, we can sample from p_{model} to generate observations that appear to have been drawn from p_{data} .
- The desirable properties of p are:
 - **Accuracy:** If $p_{model}(x)$ is high, x should look like it has been drawn from p_{data} . If $p_{model}(x)$ is low, x should *not* look like it has been drawn from p_{data} .
 - **Generation:** It should be possible to easily sample a new observation x from $p_{model}(x)$.
 - **Representation:** It should be possible to understand how different high-level features (e.g. in our example, continents) are represented by p_{model} .

P_{data} Vs P_{model}

- **Point A** is an observation that is generated by our model but does not appear to have been generated by p_{data} as it's in the middle of the sea.
- **Point B** could never have been generated by p_{model} as it sits outside the orange box.
 - model has some gaps in its ability to produce observations across the entire range of potential possibilities.
- **Point C** is an observation that could be generated by p_{model} and also by p_{data}

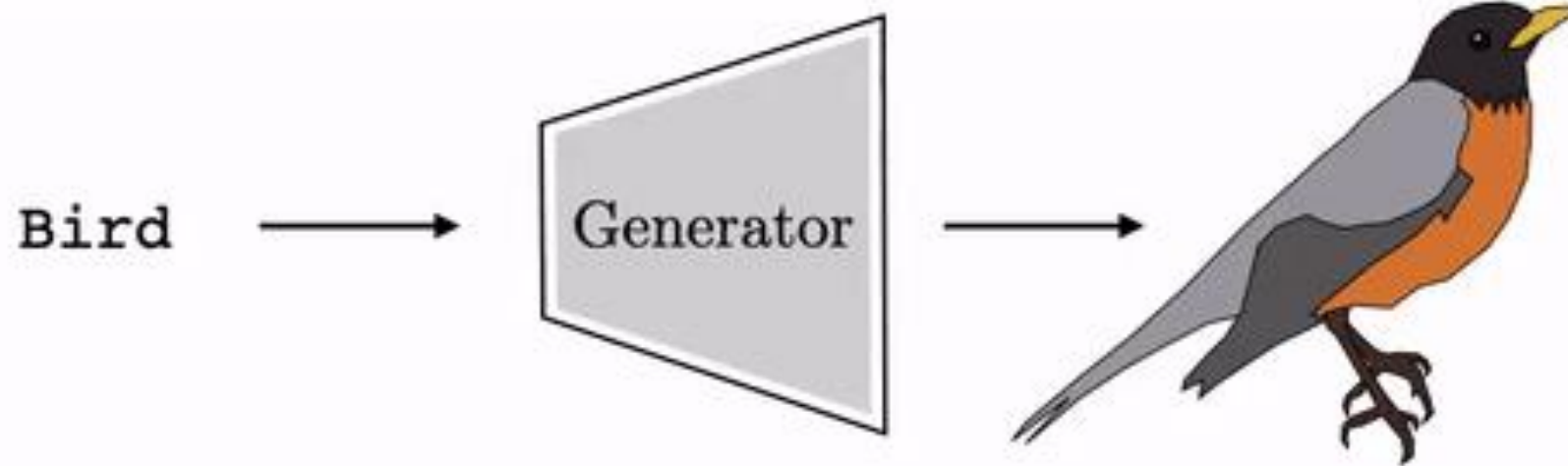


Sampling

Our learned model should be able to **make up new samples** from the distribution, **not just copy and paste** existing samples!



Sampling

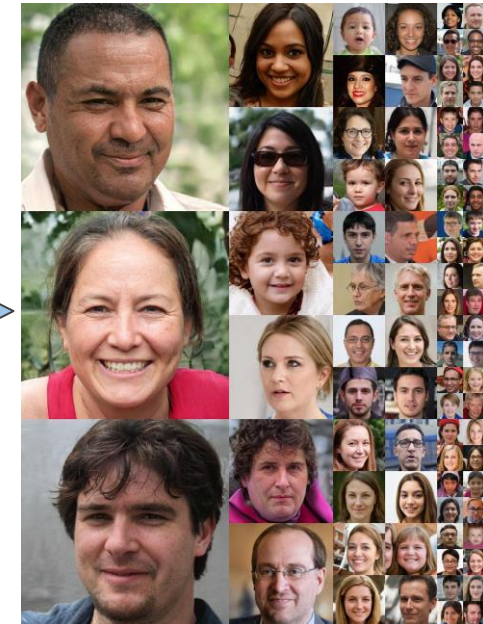
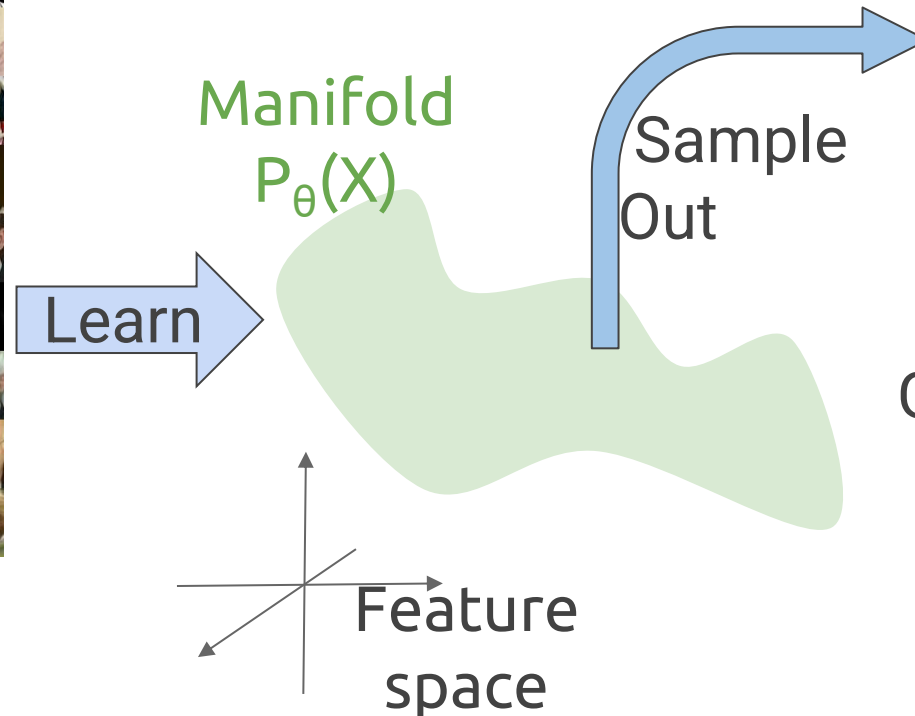


Sampling

“Model the data distribution so that we can sample new points out of the distribution”



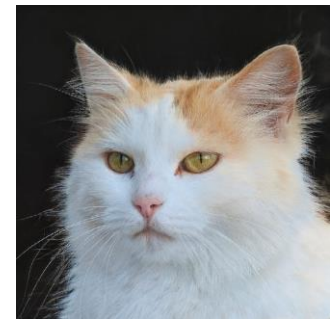
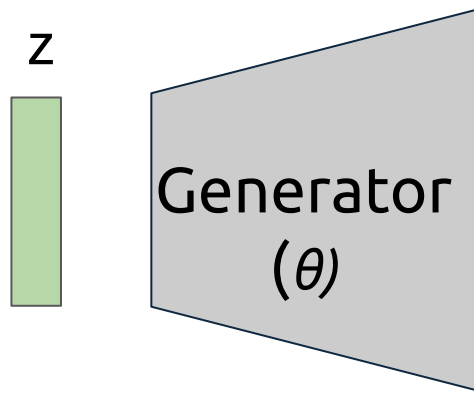
Training Dataset



Generated Samples

Sampling

How could we generate diverse samples from a deterministic deep neural network ?



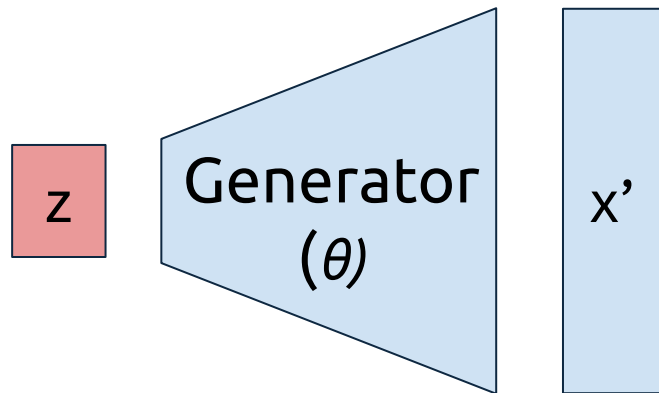
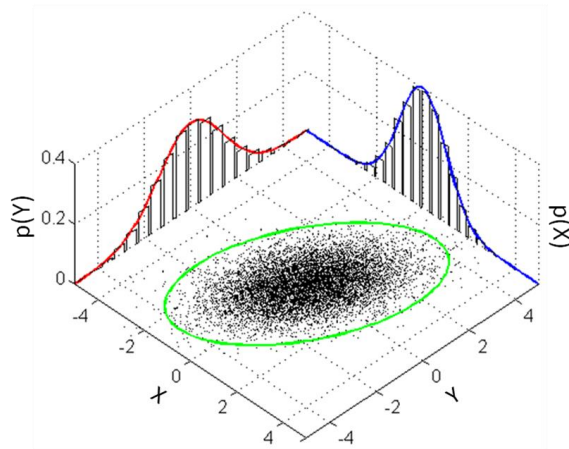
Generated Samples

Sampling

How could we generate diverse samples from a deterministic deep neural network ?

Sample z from a known prior, for example, a multivariate normal distribution $N(0, I)$.

Example: $\dim(z)=2$



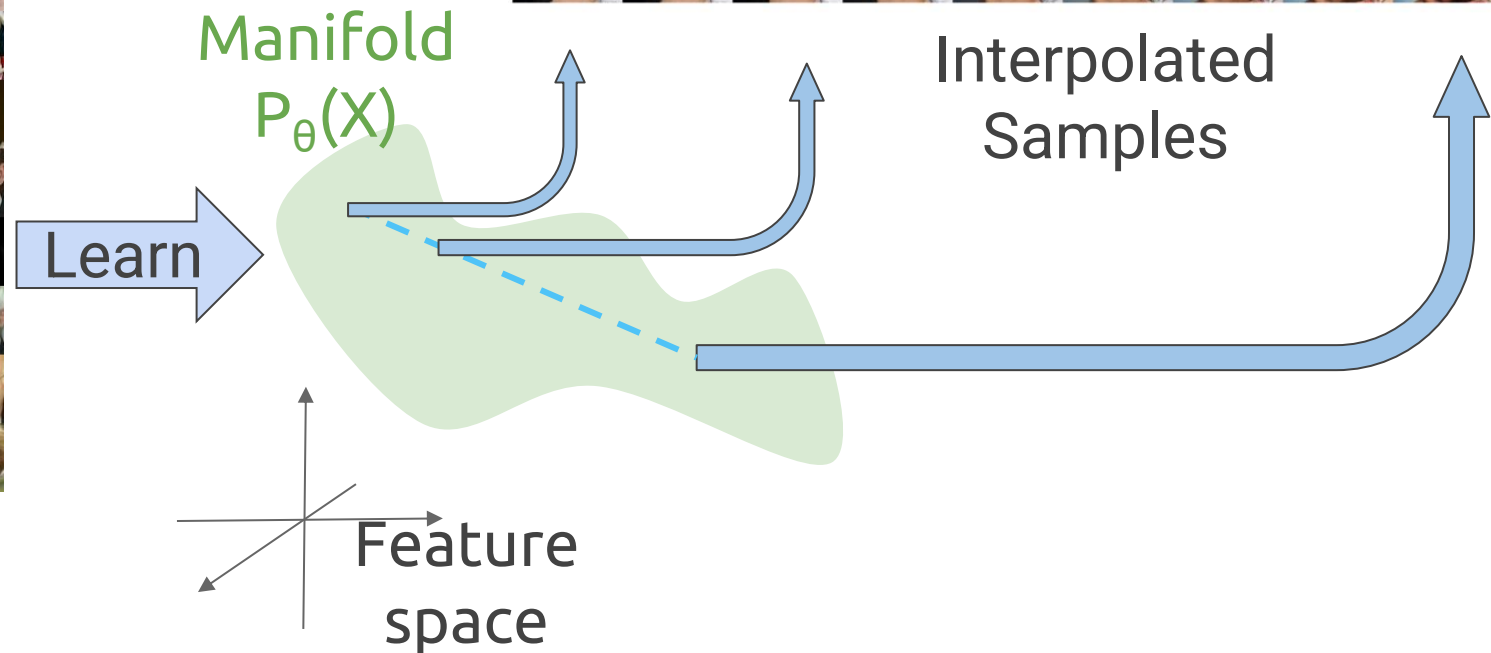
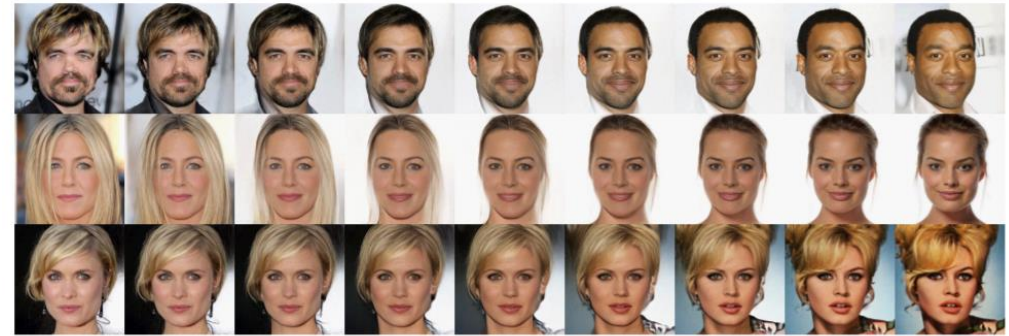
Generated Samples

Interpolation

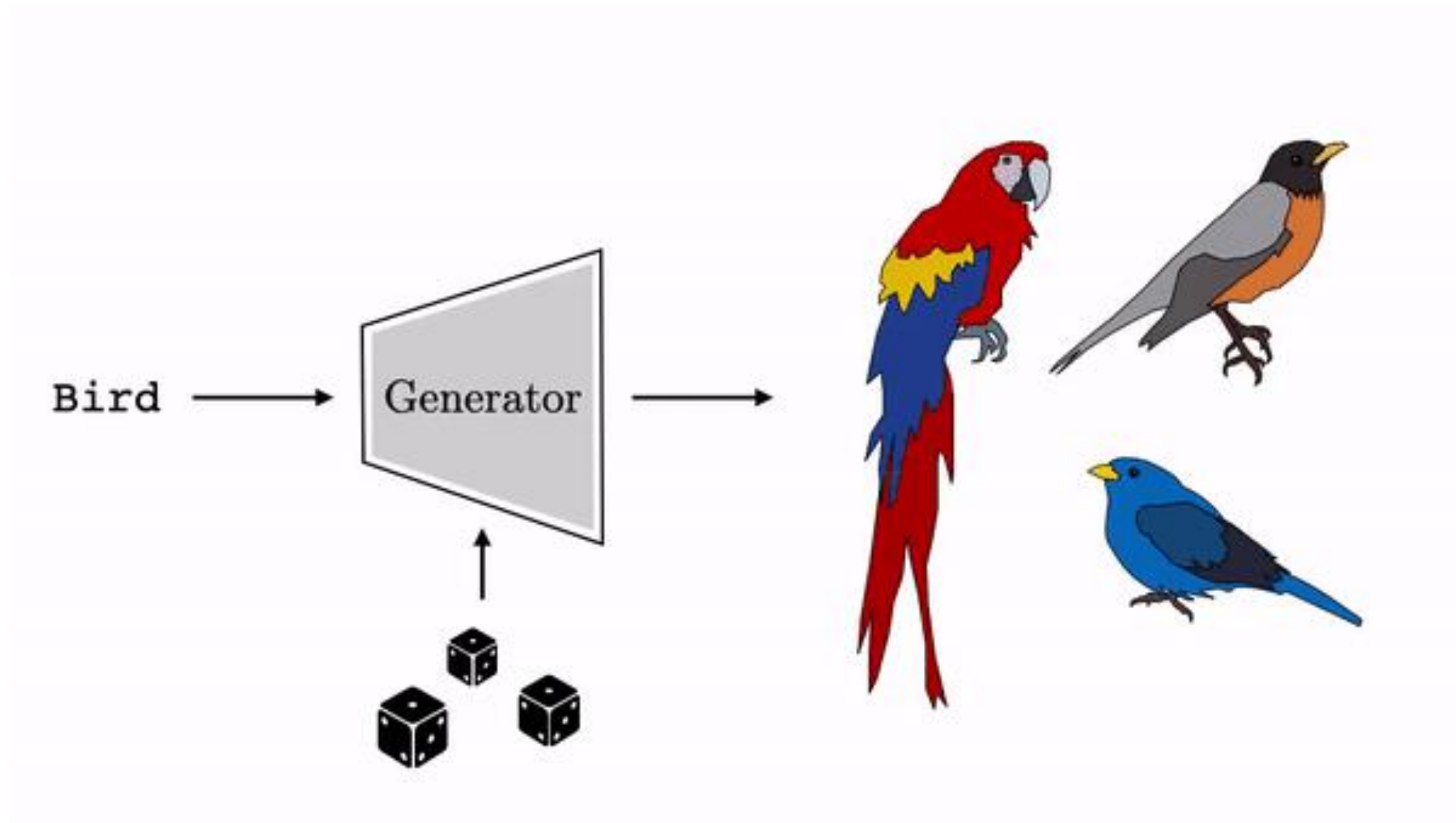
Traversing the learned manifold through interpolation.



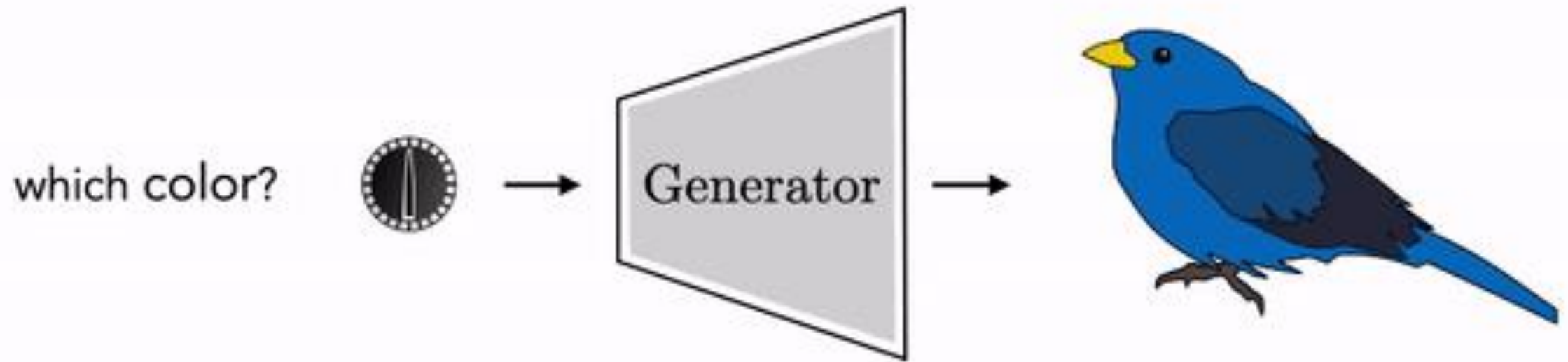
Training Dataset



Disentanglement

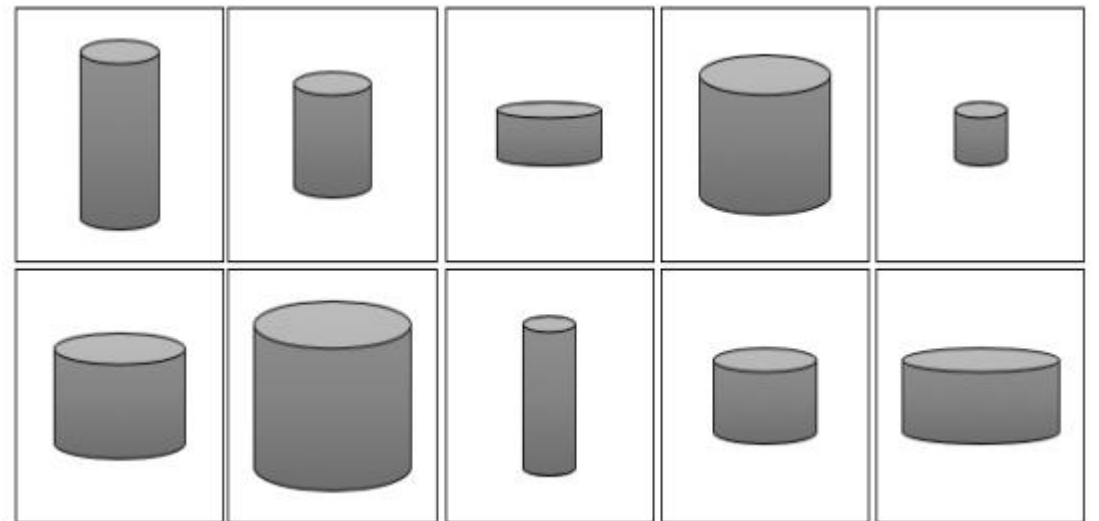


Disentanglement

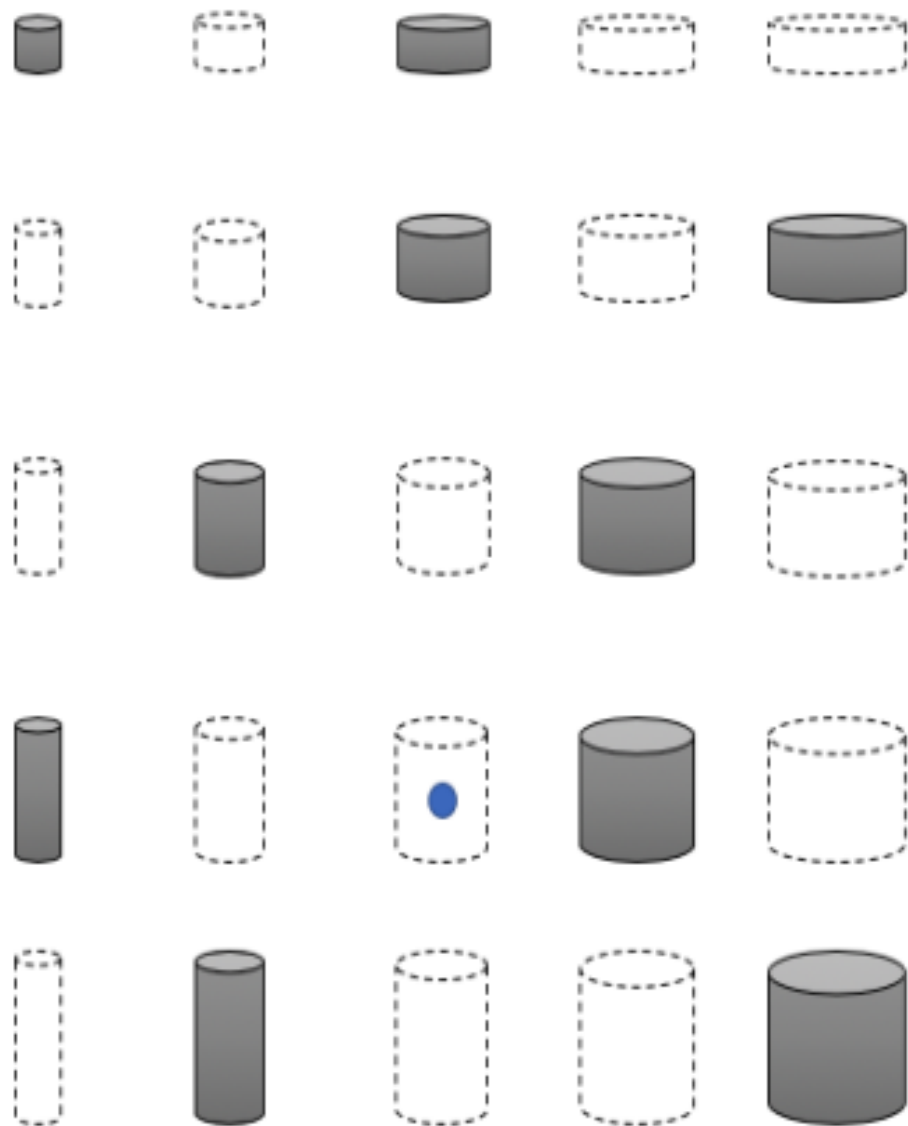


Representation Learning

- Mapping to low dimensional latent space
- Each point in latent space can be mapped to a point in original domain
- Example:
 - grayscale images of cylinders
 - Latent space: length and radius



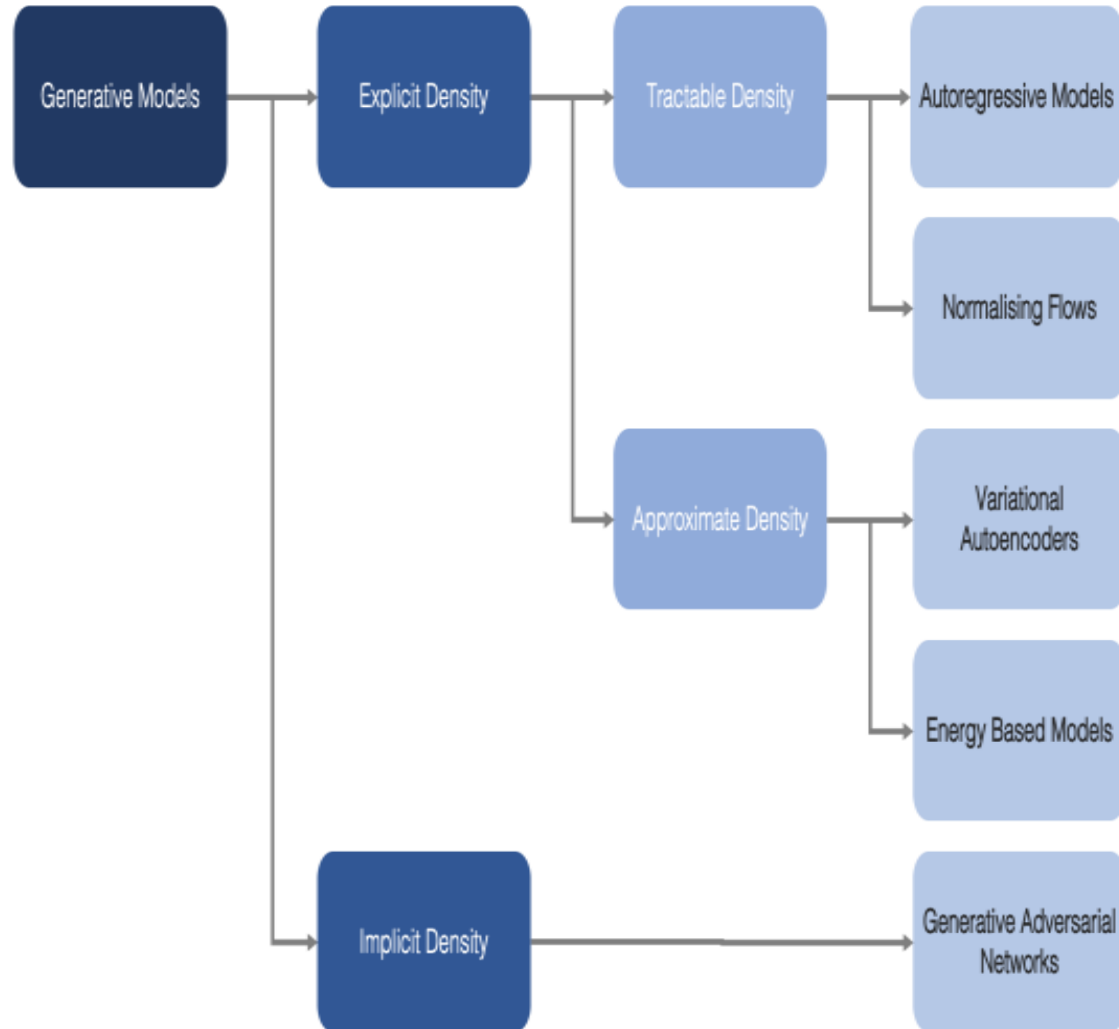
latent space



f



Generative model Taxonomy



Variational Autoencoders

Autoencoders

- Autoencoders are
 - Artificial neural networks
 - Capable of learning efficient representations of the input data, called codings, without any supervision
 - The training set is unlabeled.
- These codings typically have a much lower dimensionality than the input data, making autoencoders useful for dimensionality reduction

Autoencoders

- Useful for dimensionality reduction
- Autoencoders act as powerful feature detectors,
- And they can be used for unsupervised pre-training of deep neural networks
- Lastly, they are capable of randomly generating new data that looks very similar to the training data

Autoencoders

- Surprisingly, autoencoders work by simply learning to copy their inputs to their outputs
- This may sound like a trivial task, but we will see that constraining the network in various ways can make it rather difficult

Autoencoders

- You can limit the size of the internal representation, or you can add noise to the inputs and train the network to recover the original inputs.
- These constraints prevent the autoencoder from trivially copying the inputs directly to the outputs, which forces it to learn efficient ways of representing the data
- In short, the codings are byproducts of the autoencoder's attempt to learn the identity function under some constraints

Efficient Data Representations

Which of the following number sequences do you find the easiest to memorize?

- 40, 27, 25, 36, 81, 57, 10, 73, 19, 68
- 50, 25, 76, 38, 19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20

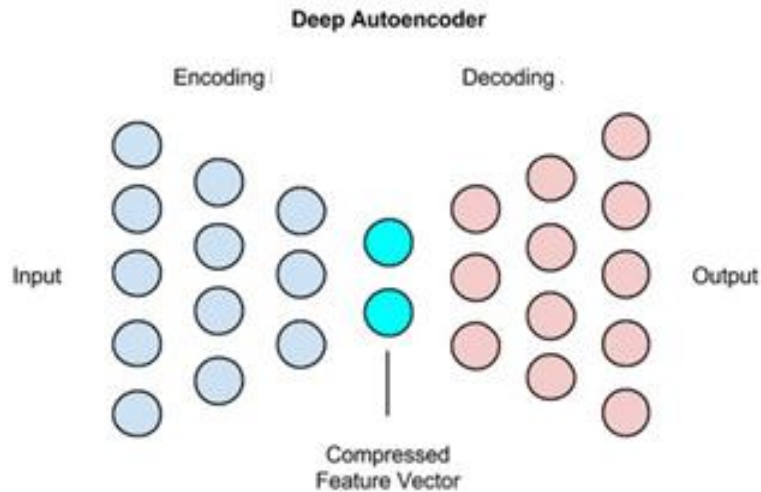
Efficient Data Representations

- At first glance, it would seem that the first sequence should be easier, since it is much shorter
- However, if you look carefully at the second sequence, you may notice that it follows two simple rules:
 - Even numbers are followed by their half,
 - And odd numbers are followed by their triple plus one
- This is a famous sequence known as the hailstone sequence

Efficient Data Representations

- Once you notice this pattern, the second sequence becomes much easier to memorize than the first because you only need to memorize the two rules,
 - The first number,
 - And the length of the sequence

Efficient Data Representations

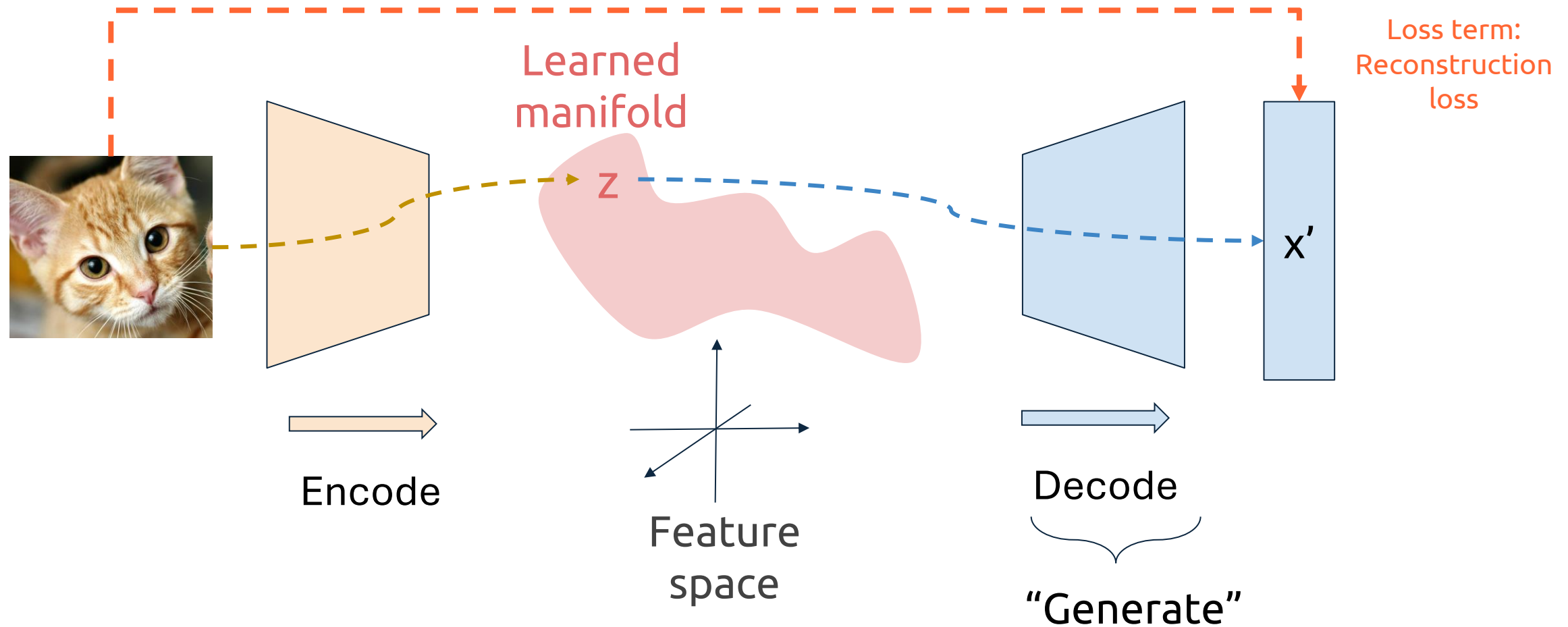


- Structure is almost always organized into **encoder** network, $\mathbf{f}$, and **decoder** network, $\mathbf{g}$: $model = \mathbf{g}(\mathbf{f}(\mathbf{x}))$
- Trained by gradient descent with **reconstruction loss**: measures differences between input and output e.g. MSE :
$$J(\theta) = |\mathbf{g}(\mathbf{f}(\mathbf{x})) - \mathbf{x}|^2$$

- Composition of Autoencoder
- An autoencoder is always composed of two parts:
 - An encoder or recognition network that converts the inputs to an internal representation,
 - Followed by a decoder or generative network that converts the internal representation to the outputs

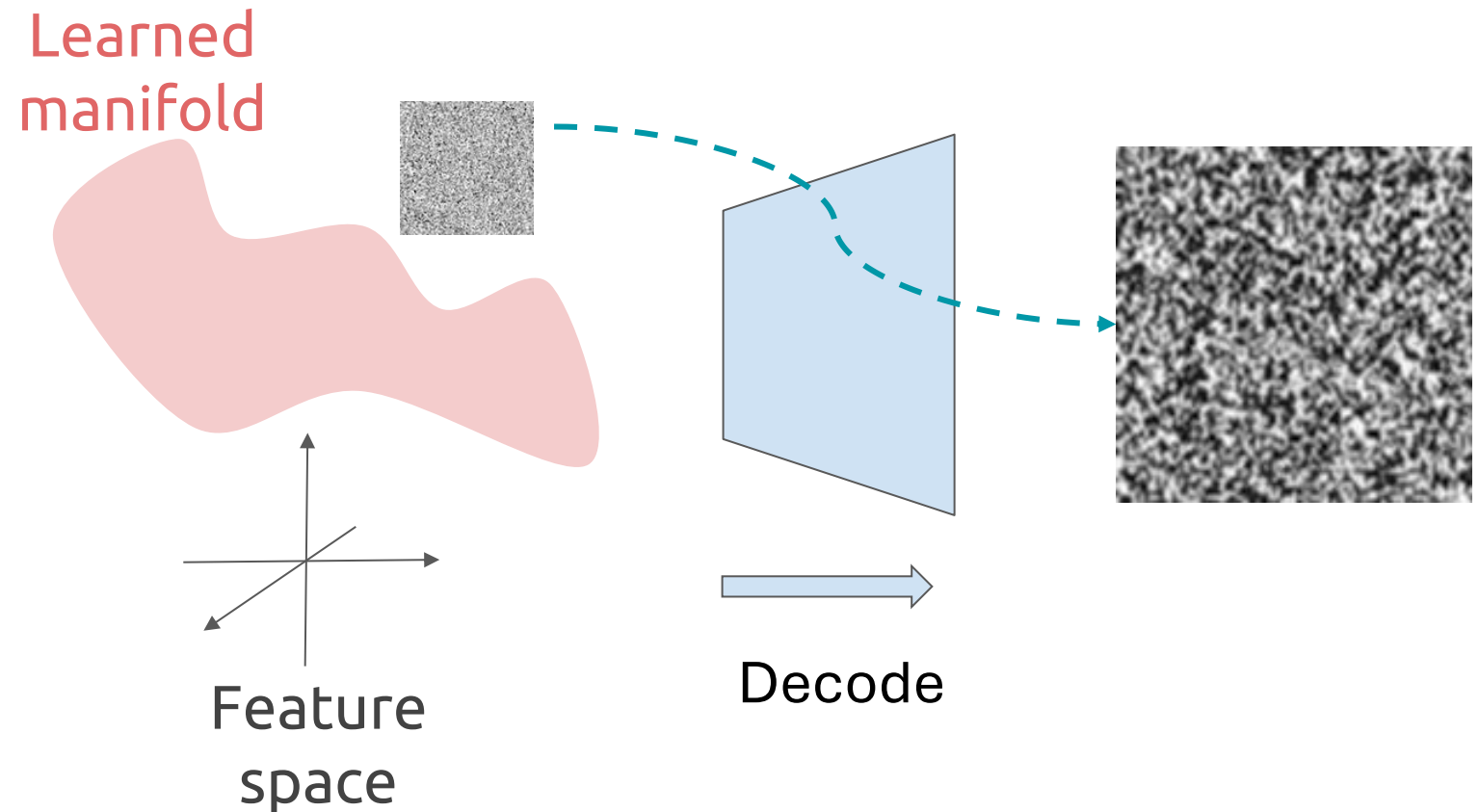
Auto-Encoder (AE)

- Trained with a reconstruction loss.
- Proposed as a pre-training stage for the encoder (“self-supervised learning”).



Auto-Encoder (AE) for generation ?

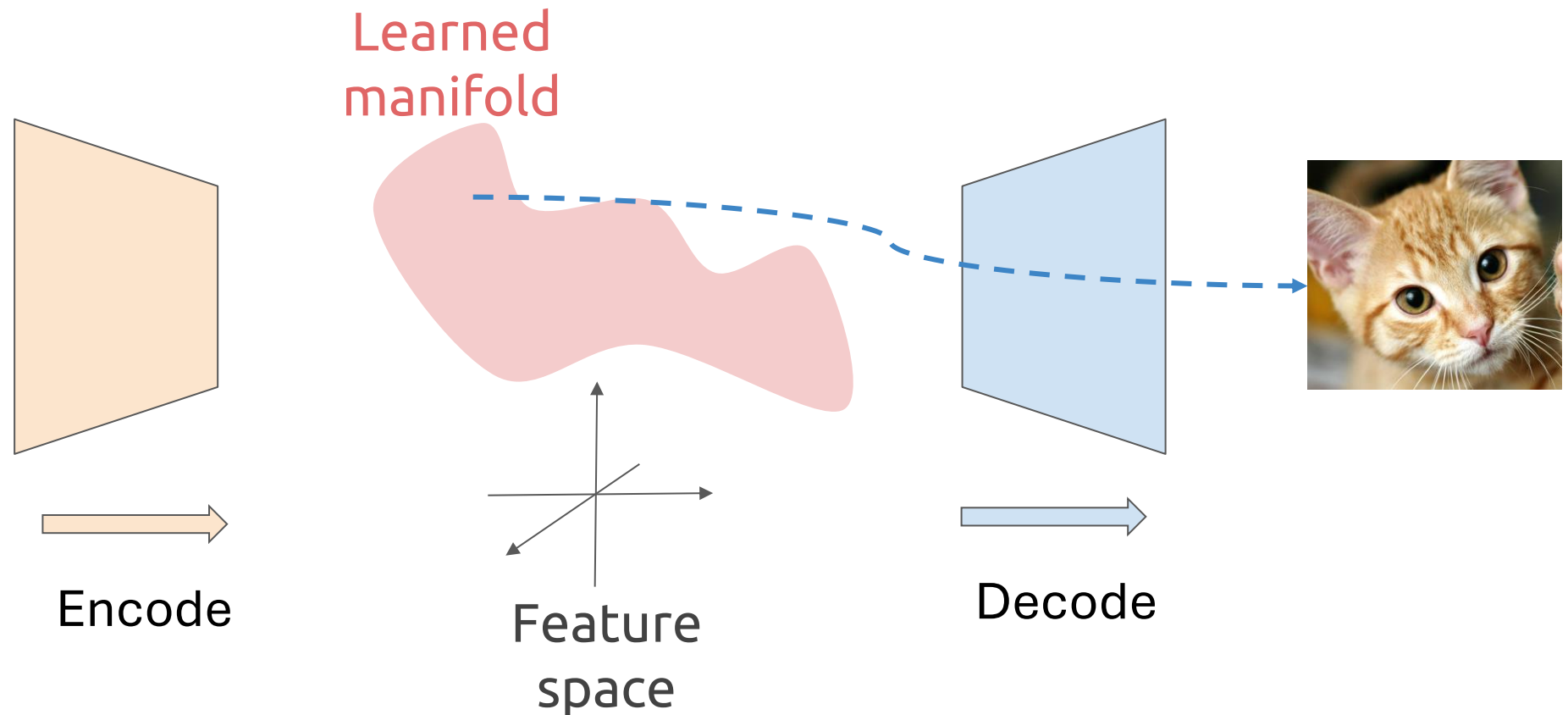
Could we generate new samples by sampling from a normal distribution and feeding it into the decoder (as in GANs) ?



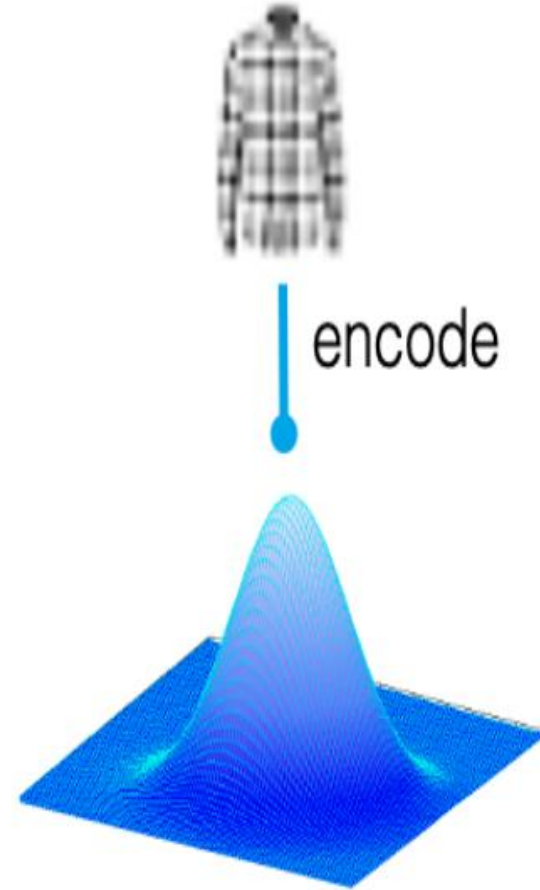
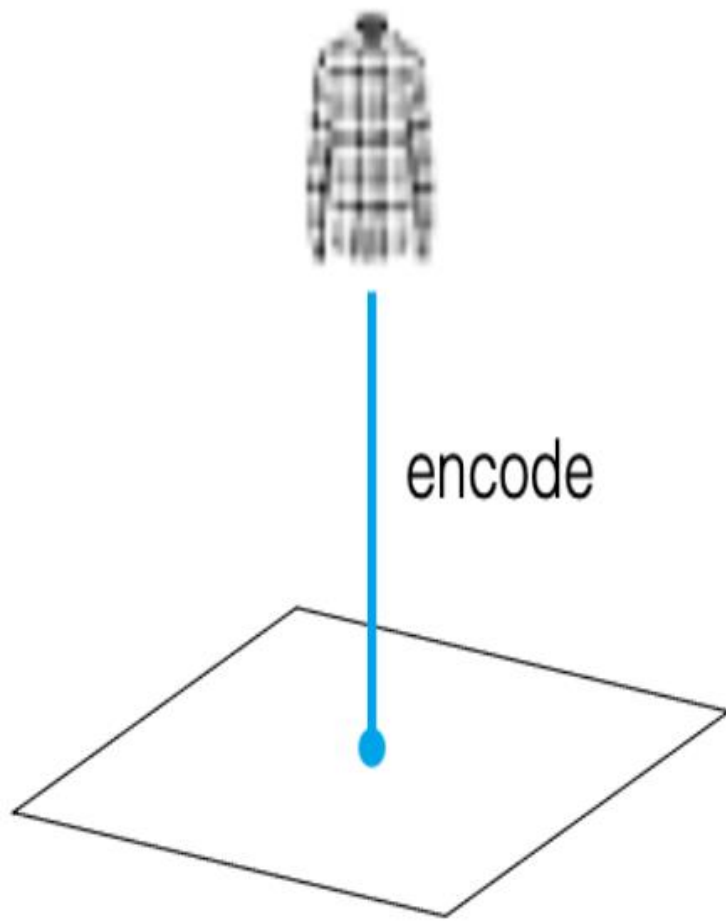
No, because the noise (or encoded noise) would be outside the learned manifold.

Auto-Encoder (AE) for generation ?

How could we train an AE that would allow to sample from its latent space ?

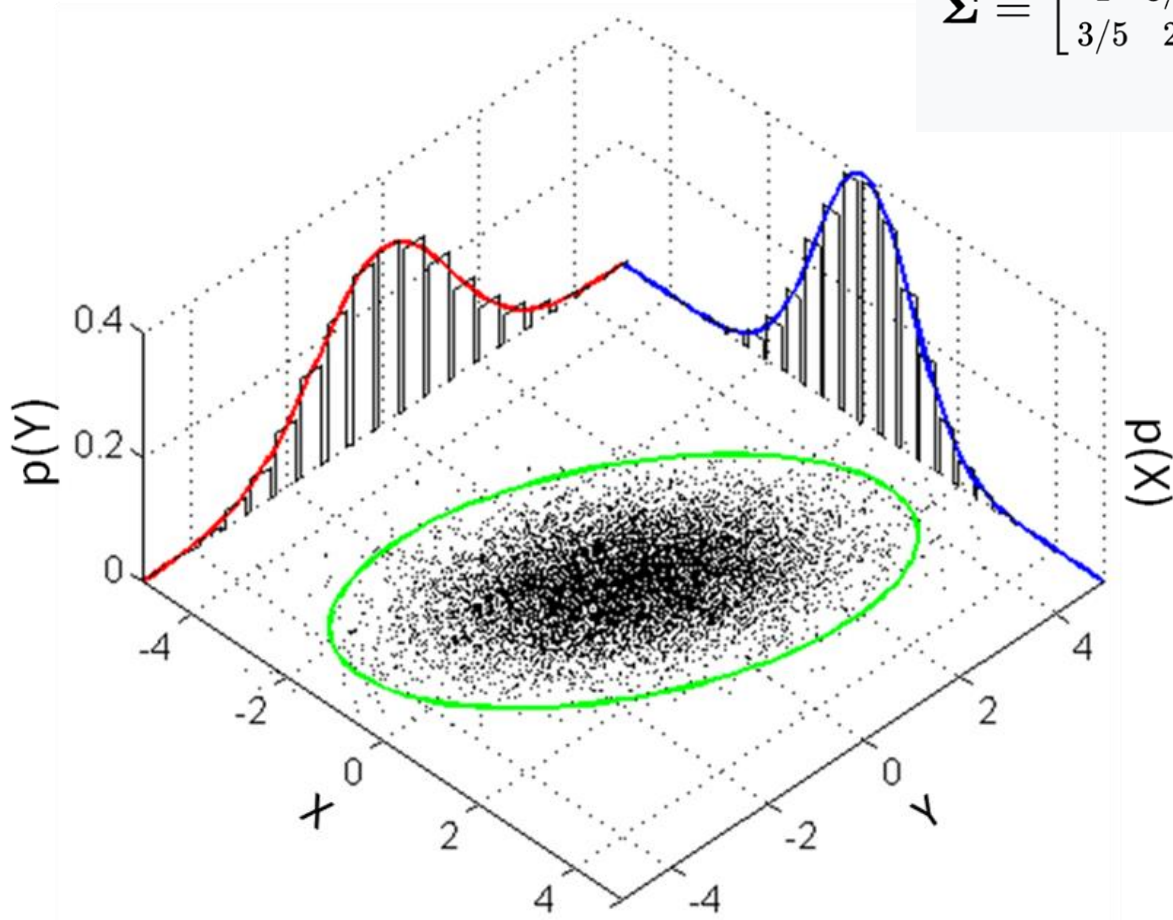


Encoder Vs Variational encoder



Maths 101: Multivariate normal distribution

Many sample points from a multivariate normal distribution with $\boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ and $\boldsymbol{\Sigma} = \begin{bmatrix} 1 & 3/5 \\ 3/5 & 2 \end{bmatrix}$, shown along with the 3-sigma ellipse, the two marginal distributions, and the two 1-d histograms.



Notation	$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$
Parameters	$\boldsymbol{\mu} \in \mathbf{R}^k$ — location $\boldsymbol{\Sigma} \in \mathbf{R}^{k \times k}$ — covariance

If all dimensions are independent:

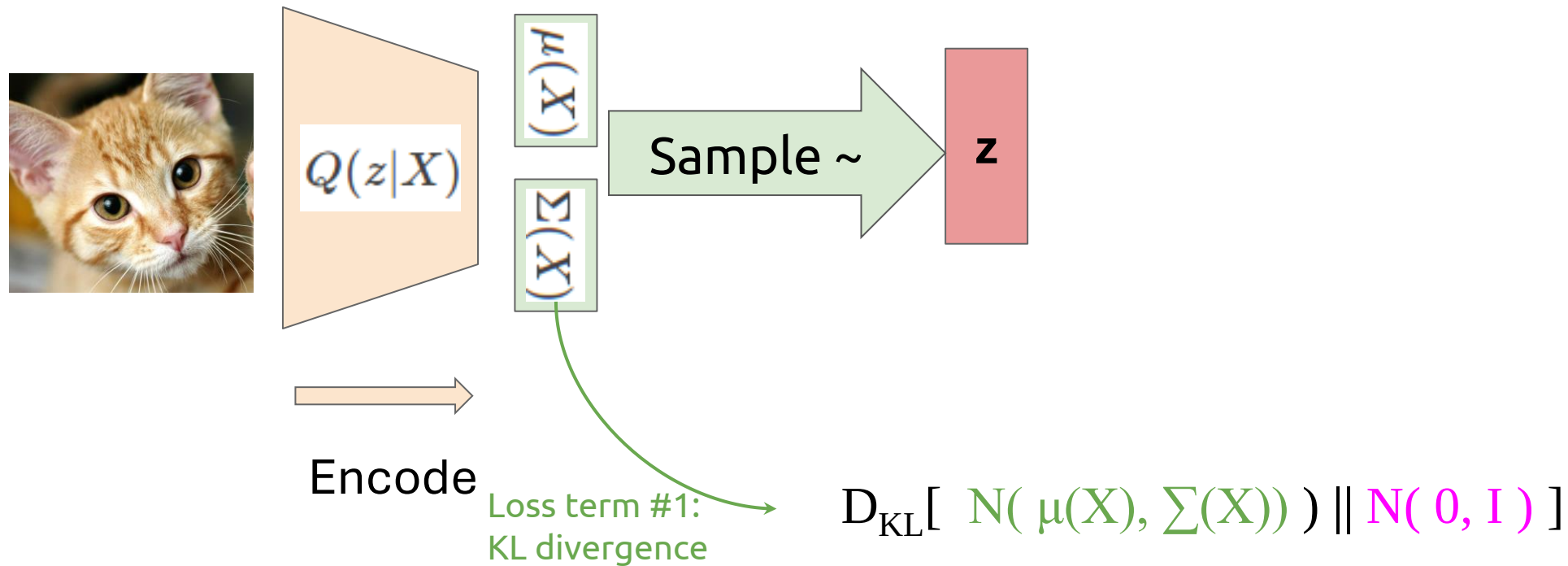
$$\boldsymbol{\Sigma} = \sigma \mathbf{I},$$

where $\sigma \in \mathbf{R}^k$ and

$\mathbf{I} \in \mathbf{R}^{k \times k}$ is the identity matrix.

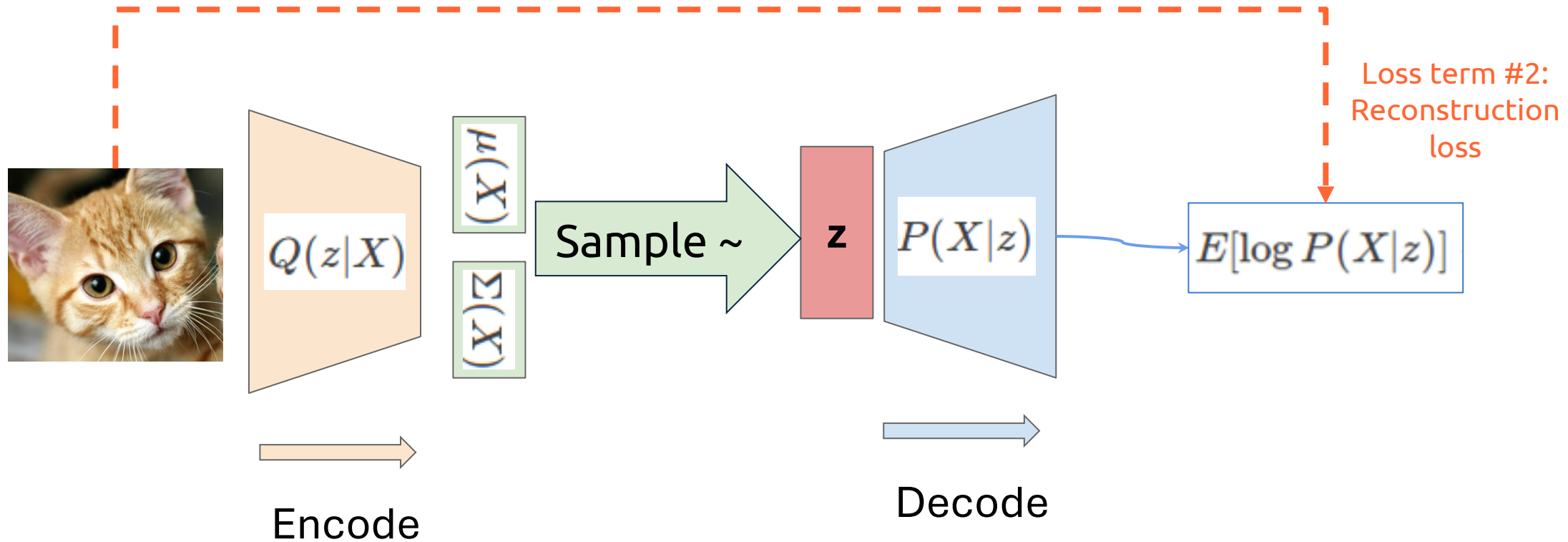
VAE - Training

Encoder (training only): Instead of mapping the input x to a fixed vector, train the encoder to map the image into it a multivariate normal distribution $N(\mu(X), \Sigma(X))$ that should map a standard normal distribution $N(0, I)$.



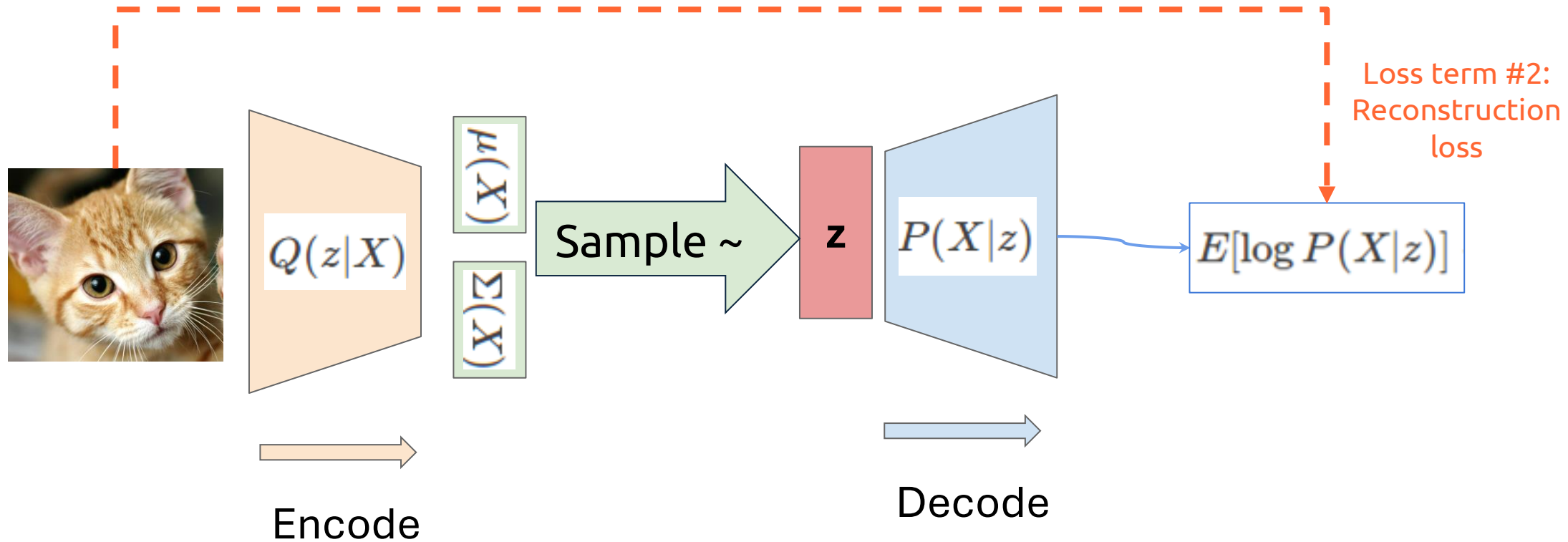
VAE - Training

Decoder: Reconstruct the input data from $z \sim N(\mu(X), \Sigma(X))$.



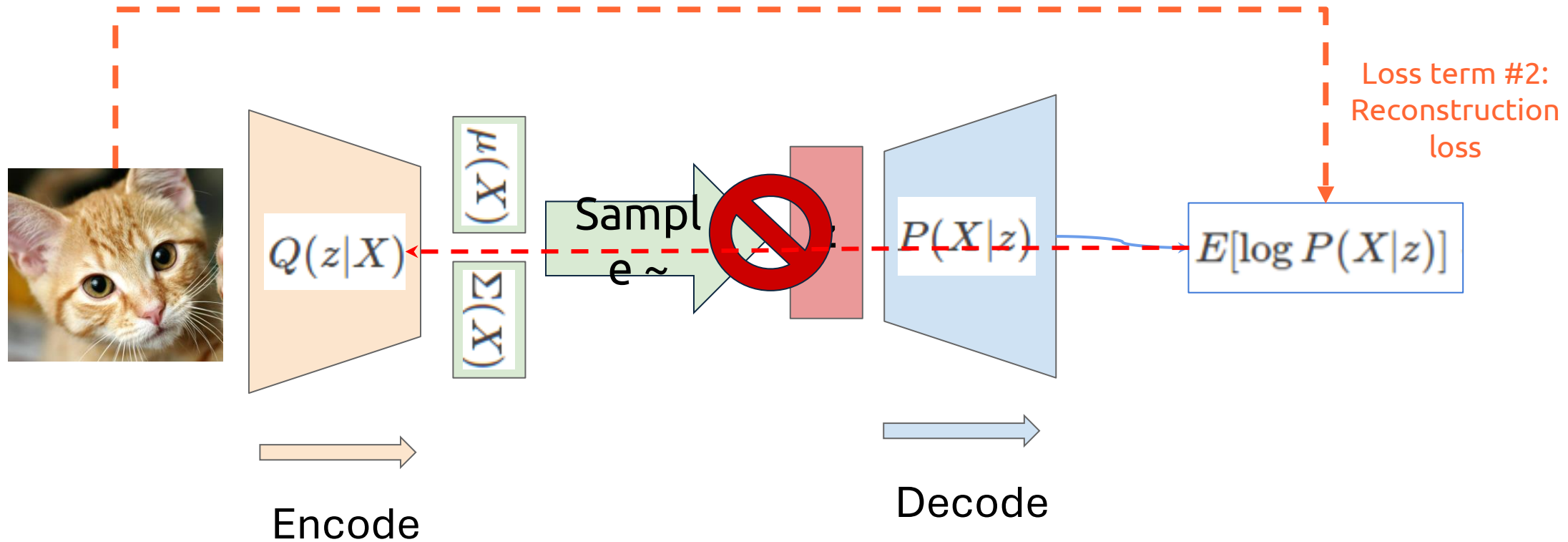
VAE - Training

Do you see any challenge in this scheme if use deep neural networks to estimate $Q(z|X)$ and $P(X|z)$?



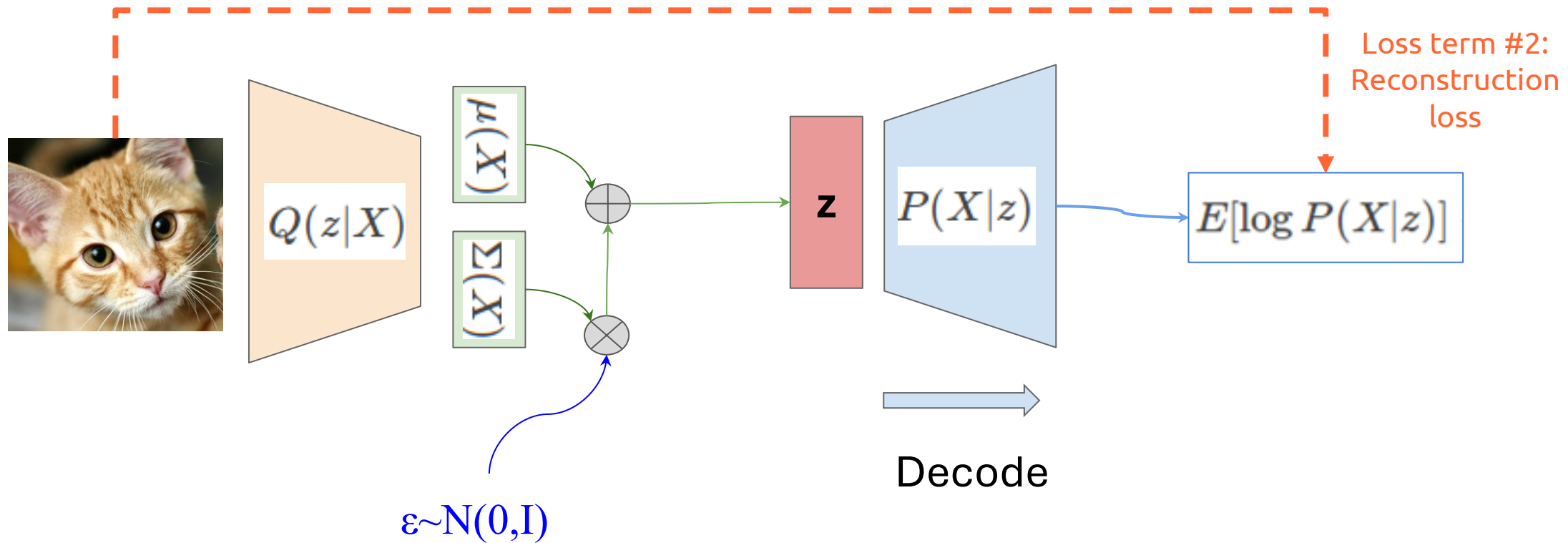
VAE - Training

Challenge: We cannot backprop through “Sampling” because it is not a differentiable operation.



VAE - Training

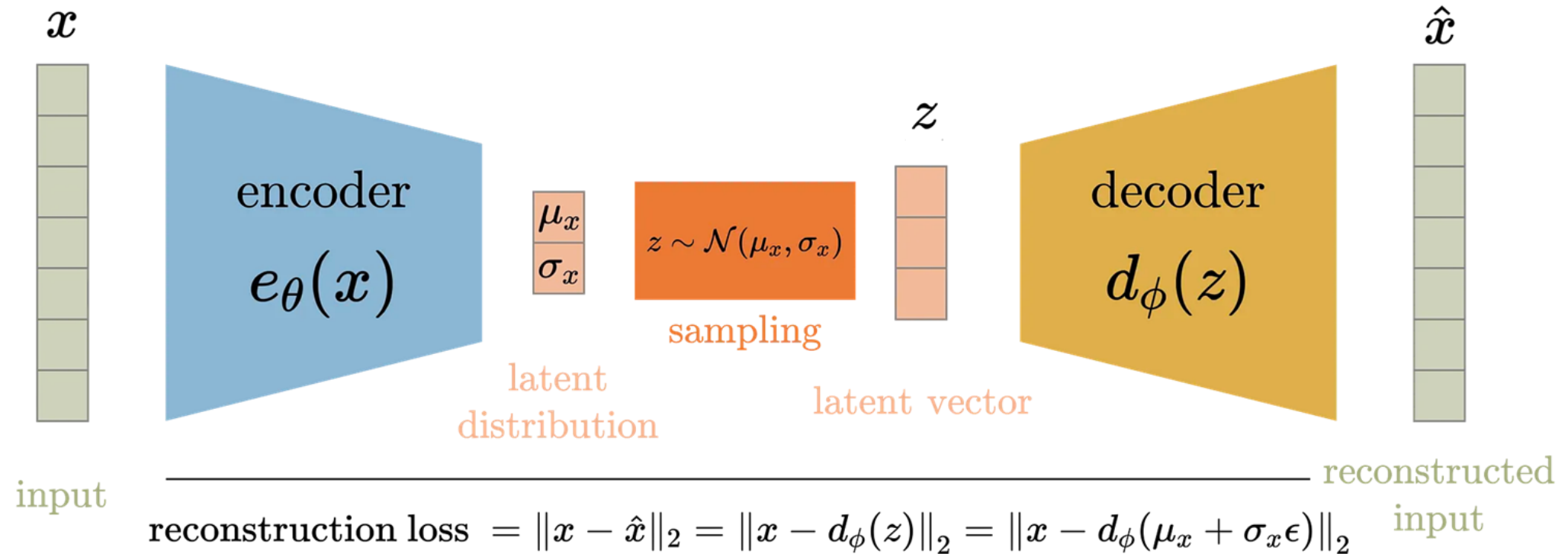
Sampling: The reparametrization trick uses an auxiliary variable $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ to obtain a sample z from the distribution $\mathcal{N}(\mu, \Sigma)$.



VAE - Training

Figure: [Aqeel Anwar](#)

In practice, as we want to train $\mathcal{N}(\mu(X), \Sigma(X))$ to become $\mathcal{N}(0, \mathbf{I})$, the encoder predicts a vector $\sigma(X)$ instead of a matrix $\Sigma(X)$.



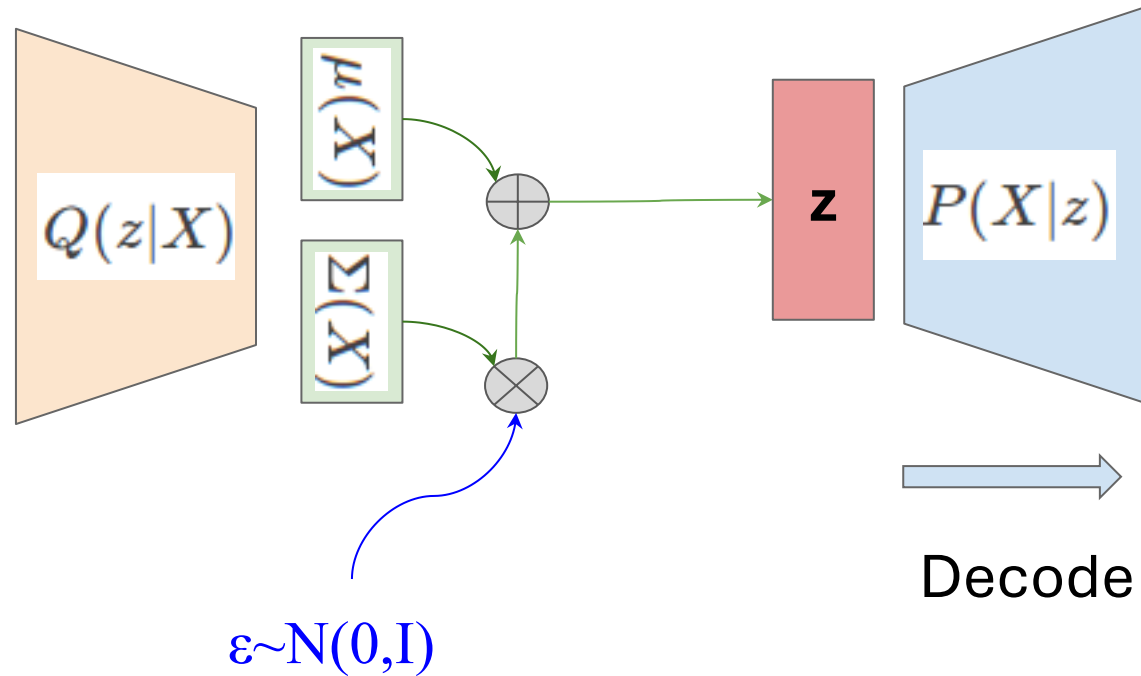
$$\mu_x, \sigma_x = e_{\theta}(x), \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\text{similarity loss} = KL \text{ Divergence} = D_{KL}(\mathcal{N}(\mu_x, \sigma_x) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))$$

$$\text{loss} = \text{reconstruction loss} + \text{similarity loss}$$

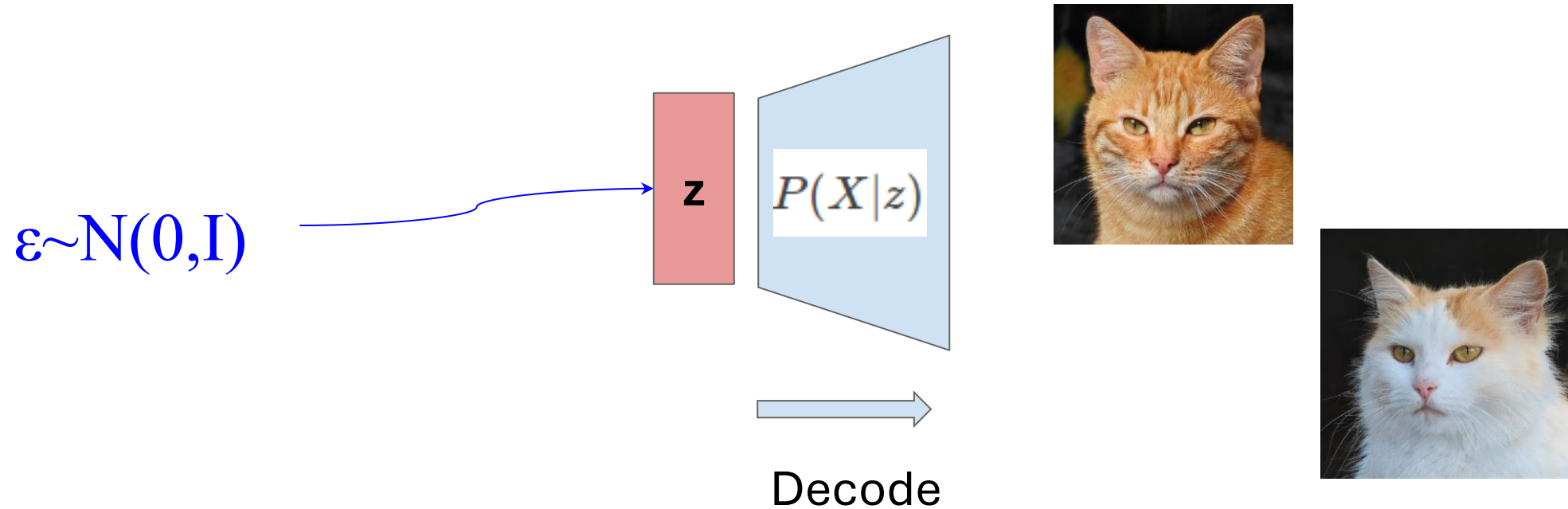
VAE Inference

How should be modify the training scheme to generate new samples from a trained VAE ?



Generative behaviour

We can sample from our prior $\epsilon \sim \mathcal{N}(0, I)$, discarding the encoder path.



Generative behaviour

